Católica de Santa Catarina
Análise Preditiva
Prof. M. Sc. Rodrigo Hagstrom

Trabalho Avaliativo N3
Pipeline MLOps para prever a temperatura máxima do dia seguinte em Jaraguá do Sul

Relatório técnico

André Ramos da Silva
Artur Felippe Zanella
Felipe Bogo
Jeison Diniz de Souza Marinho
Thierry Augusto Busnardo

Repositório: github.com/zanelladev/n3-mlops-temperatura-jaragua
Jaraguá do Sul, novembro de 2026

Visão geral

Este relatório descreve o projeto que fizemos para a N3: um pipeline de MLOps que, todo dia, busca dados novos de clima, escolhe as features por votação de três métodos, coloca dois algoritmos para competir e decide sozinho se o modelo em produção deve ser trocado. O problema é prever a temperatura máxima do dia seguinte em Jaraguá do Sul. Escolhemos clima porque a fonte é real, gratuita e muda todo dia, então a ingestão e o versionamento dos dados acontecem de verdade, sem simulação. Até a geração deste relatório (08/11/2026) o workflow fechou 3 ciclos completos, o último com dados até 01/11/2026. Todos os números abaixo saem das evidências que o próprio workflow grava no repositório, e o script `reports/gerar_relatorio.py` refaz este PDF a partir delas.

Problema e dados

Problema. Regressão. A variável-alvo é a temperatura máxima do dia t+1, em °C, e a unidade de observação é um dia em Jaraguá do Sul. Tudo o que o modelo usa é conhecido até o fim do dia t.

Fonte e licença. Historical Weather API do Open-Meteo, com a reanálise ERA5-Seamless (combinação do ERA5 e do ERA5-Land, do Copernicus/ECMWF). A API é gratuita, não pede chave, e os dados são distribuídos sob a licença CC BY 4.0, que permite o uso com atribuição ("Weather data by Open-Meteo.com"). Usamos o ponto de grade mais próximo de Jaraguá do Sul (-26,49; -49,07), com a série diária desde 01/01/2010 agregada no fuso de São Paulo. No último ciclo a base tinha 6.149 dias, de 01/01/2010 a 01/11/2026, em 3 lotes.

Atributos de origem. Treze variáveis diárias: temperatura máxima, temperatura mínima, temperatura média, umidade relativa média, ponto de orvalho médio, pressão média ao nível do mar, nebulosidade média, vento máximo, vento médio, rajada máxima, direção dominante do vento, chuva acumulada, radiação solar. Sobre elas construímos os atributos da seção de ETL, chegando a 26 candidatas.

Limitações dos dados. Reanálise não é estação meteorológica: é um modelo que assimila observações numa grade de 9 a 25 km, então suaviza os extremos locais. O ERA5 chega com cerca de 7 dias de atraso, por isso o "dia seguinte" é o dia seguinte ao último dia disponível, e não o amanhã do calendário. É uma série de um ponto só, e os dias mais recentes do ERA5 podem ser levemente revisados depois; guardamos o valor da primeira consulta para manter cada versão reproduzível.

Métrica e desenho da avaliação

Métrica principal. MAE em °C, a minimizar, fixada no `config.yaml` antes do primeiro treino. Ela está na unidade do problema, não deixa poucos dias atípicos dominarem

a comparação como o RMSE faria, e é fácil de comparar com a referência ingênua da persistência (a máxima de amanhã igual à de hoje). RMSE e R^2 são registrados como apoio.

Particionamento. Como é série temporal, nada é embaralhado. Com os dias em ordem: treino, depois a validação com os 365 dias seguintes e por fim o teste com os 30 dias mais recentes. A validação tem um ano inteiro para que a comparação entre modelos passe por todas as estações. No último ciclo as janelas foram: treino de 07/01/2010 a 01/10/2025 (5.747 dias), validação de 02/10/2025 a 01/10/2026 e teste de 02/10/2026 a 31/10/2026. Votação, escolha entre A e B e promoção usam só treino e validação; o teste fica reservado para a avaliação final do modelo vigente.

Chegada de novos registros. Cada execução pergunta à API pelos dias depois do último que já está em `data/raw` e grava o que vier num arquivo novo, `lote_<inicio>_<fim>.csv`. Como as janelas são contadas a partir do fim da série, um dia novo entra no teste e empurra um dia da validação para o treino.

Por que o vigente nunca é retreinado com validação e teste. O modelo promovido é exatamente o que foi avaliado, treinado só com a janela de treino. Assim ele nunca viu os dias de validação dos ciclos seguintes (as janelas só andam para frente), e a comparação com cada novo desafiante é justa. O notebook 04 confere isso e para o pipeline se não for verdade. O custo é o modelo não aprender com os últimos 13 meses, o que pesa pouco diante de 15 anos de histórico.

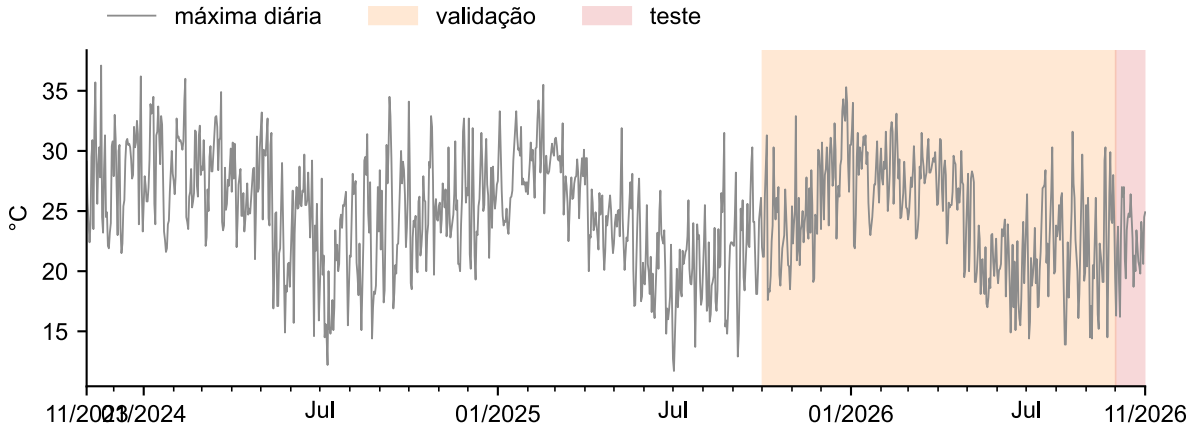


Figura 1. Máxima diária nos últimos três anos da série, com as janelas de validação e teste do último ciclo.

ETL e engenharia de atributos

O notebook 01 junta os lotes, faz o diagnóstico, trata, valida e monta os atributos. O diagnóstico procura ausentes, datas duplicadas, dias faltando no calendário, valores fora de uma faixa física plausível para a região (por exemplo, umidade fora de 0 a 100% ou pressão fora de 950 a 1050 hPa), mínima maior que a máxima e outliers

por z robusto acima de 5. A Tabela 1 mostra o diagnóstico do último ciclo, antes e depois dos tratamentos.

Tabela 1. Diagnóstico por variável, antes e depois da limpeza (último ciclo).

	antes			depois		
variável	**ausentes**	**fora da faixa**	**outliers (z > 5)**	**ausentes**	**fora da faixa**	**outliers (z > 5)**
tmax	0	0	0	0	0	0
tmin	0	0	0	0	0	0
tmed	0	0	0	0	0	0
umidade	0	0	22	0	0	22
orvalho	0	0	0	0	0	0
pressao	0	0	0	0	0	0
nuvens	0	0	0	0	0	0
vento_max	0	0	9	0	0	9
vento_med	0	0	40	0	0	40
rajada	0	0	15	0	0	15
vento_dir	0	0	0	0	0	0
chuva	0	0	1026	0	0	1026
radiacao	0	0	0	0	0	0

Resumo geral antes da limpeza: 6.149 linhas, 0 datas duplicadas, 0 dias faltando e 0 casos de mínima maior que a máxima. A reanálise vem limpa; o que o diagnóstico mais acusa são outliers pelo z robusto, principalmente em `chuva` (1.026), `vento_med` (40), `umidade` (22), `rajada` (15). São variáveis assimétricas (a maioria dos dias tem pouca ou nenhuma chuva), então os valores altos são dias de chuva forte ou de vento forte de verdade, e ficam na base. Os tratamentos, todos no código e não à mão, são:

- **Data repetida:** fica a versão do lote mais novo, então reprocessar um lote nunca duplica dias.
- **Dia faltando:** vira uma linha vazia, para os lags e as médias móveis sempre olharem para o dia anterior de verdade.
- **Valor fisicamente impossível ou mínima maior que a máxima:** vira ausente. A imputação pela mediana acontece dentro do Pipeline, ajustada só com o treino, junto com a padronização. Nada é ajustado com validação ou teste.
- **Outlier:** só sinalizado. Em clima, uma onda de calor ou uma frente fria forte são dados reais, e são justamente os dias difíceis de prever.

Depois dos tratamentos uma validação para o pipeline se faltar coluna, se sobrar data repetida ou buraco no calendário, ou se mais de 1% do alvo estiver ausente. Os atributos construídos olham só para trás:

- `tmax_lag1` a `tmax_lag3`, `tmax_media3` e `tmax_media7`: a temperatura tem muita inércia.
- `tmax_delta` e `amplitude`: se esquentou ou esfriou de ontem para hoje, e a diferença entre máxima e mínima (grande em dia de céu aberto).
- `pressao_delta`, `umidade_delta` e `chuva_3d`: pressão caindo e umidade subindo costumam anunciar frente fria e chuva.
- `vento_dir_sin` e `vento_dir_cos`: direção do vento é circular (350° fica perto de 10°); vento de norte esquenta, vento de sul esfria.
- `dia_ano_sin` e `dia_ano_cos`: a época do ano, sem o salto entre 31/12 e 01/01.

Versão dos dados. O `data_version` é o último dia da série mais o início do sha256 dos arquivos de `data/raw`, por exemplo `dv-20261101-158b26fd`. Mesmo dado gera a mesma versão; qualquer dia novo gera outra. Conferimos isso rodando os mesmos dois primeiros ciclos na nossa máquina e no runner do GitHub: os `data_version` saíram idênticos. O `ciclo.json` de cada ciclo lista os lotes que existiam naquele momento.

Seleção de features por votação

O notebook 02 roda os três métodos no mesmo notebook, todos ajustados só com o treino: SelectKBest com `f_regression` (teste F de cada atributo contra o alvo, adequado para alvo contínuo), importância do Random Forest (200 árvores, pega relações não lineares) e RFE com Ridge (tira o atributo de menor coeficiente até sobrarem k, então descarta os redundantes). Não usamos validação cruzada, então não existe seleção dentro de fold: a validação e o teste simplesmente não entram nesta etapa.

Regra de votação, fixada no config antes de rodar: cada método vota nas suas k = 10 melhores; é eleita a feature com pelo menos 2 dos 3 votos; se a maioria eleger menos de 6, o conjunto é completado pelo menor ranking médio dos três métodos, que também desempata a ordem. Escolhemos maioria porque um atributo forte em só um método pode ser artefato daquele método (o F-score, por exemplo, gosta de todas as variáveis de temperatura ao mesmo tempo, mesmo sendo quase cópias umas das outras).

Tabela 2. Votação do último ciclo (20261108-170654). Rank 1 é o melhor de cada método; F = SelectKBest, RF = Random Forest.

feature	rank F	rank RF	rank RFE	voto F	voto RF	voto RFE	votos	ranking médio	eleita
tmed	1	1	1	•	•	•	3	1,0	**sim**
dia_ano_cos	7	4	3	•	•	•	3	4,7	**sim**
tmin	4	6	8	•	•	•	3	6,0	**sim**
radiacao	8	10	4	•	•	•	3	7,3	**sim**
vento_dir_cos	13	3	2		•	•	2	6,0	**sim**
tmax	2	2	19	•	•		2	7,7	**sim**
pressao_delta	14	8	9		•	•	2	10,3	**sim**
rajada	25	5	6		•	•	2	12,0	**sim**
orvalho	3	13	11	•			1	9,0	
pressao	9	12	16	•			1	12,3	
amplitude	17	14	7			•	1	12,7	
tmax_media7	6	21	12	•			1	13,0	
chuva	20	9	13		•		1	14,0	
tmax_media3	5	23	14	•			1	14,0	
nuvens	24	15	5			•	1	14,7	
vento_dir_sin	19	7	22		•		1	16,0	
tmax_lag1	10	25	20	•			1	18,3	
umidade	21	26	10			•	1	19,0	
tmax_lag2	11	22	17				0	16,7	
tmax_lag3	12	18	21				0	17,0	
dia_ano_sin	16	11	24				0	17,0	
tmax_delta	15	16	25				0	18,7	
vento_med	18	20	23				0	20,3	
chuva_3d	23	24	15				0	20,7	
umidade_delta	26	19	18				0	21,0	
vento_max	22	17	26				0	21,7	

Foram eleitas 8 features: `tmed`, `dia_ano_cos`, `tmin`, `radiacao`, `vento_dir_cos`, `tmax`, `pressao_delta`, `rajada`. 4 tiveram os três votos e 4 tiveram dois, então a regra de mínimo não precisou entrar. Um resultado que chamou atenção: `tmax_lag1`, `tmax_lag2`, `tmax_lag3` ficaram de fora. A informação dos dias anteriores já aparece em `tmed`, `tmax` e `tmin` do próprio dia, e os métodos que olham os atributos em conjunto prefe-

rem ficar com uma dessas variáveis em vez de várias quase iguais. Só o F-score, que avalia uma variável de cada vez, chegou a votar em algum lag. A votação é refeita em todo ciclo, e nos 3 ciclos o conjunto eleito foi sempre o mesmo. Votos, rankings, importâncias, parâmetros da regra e `cycle_id` ficam no run `votacao` do MLflow.

Competição entre os dois modelos

O notebook 03 treina dois algoritmos distintos com as mesmas features eleitas, o mesmo treino, o mesmo Pipeline de pré-processamento (imputação pela mediana e padronização) e a mesma semente (42), e avalia os dois na mesma janela de validação:

- **Modelo A, Ridge** (alpha = 1.0): regressão linear com penalização L2, simples e estável com atributos parecidos entre si.
- **Modelo B, HistGradientBoosting** (300 iterações, taxa 0,05, até 15 folhas por árvore): árvores em sequência, cada uma corrigindo o erro das anteriores, capaz de pegar efeitos não lineares como o da frente fria.

Tabela 3. Métricas na validação (02/10/2025 a 01/10/2026), último ciclo.

modelo	MAE (°C)	RMSE (°C)	R²
A: ridge	1,945	2,460	0,702
B: hist_gradient_boosting	1,897	2,417	0,713
persistência (amanhã = hoje)	2,610	3,385	0,436

Os dois modelos erram bem menos que a persistência: o melhor tem MAE 27% menor. Entre os dois, a diferença é pequena (0,048 °C no último ciclo): o modelo B venceu em 3 de 3 ciclos. Para cada modelo o MLflow guarda hiperparâmetros, semente, lista de features, janelas, métricas e o modelo serializado (o Pipeline inteiro, no formato skops), num run identificado pelo mesmo `cycle_id` e `data_version` da votação.

Eleição do vencedor e promoção

O notebook 04 elege o vencedor do ciclo pelo menor MAE na validação (empate fica com A, o mais simples). Depois carrega o modelo vigente pelo MLflow, a partir do manifesto `models/modelo_vigente.json`, e avalia esse modelo na mesma janela de validação, com as features dele. A regra de promoção, definida antes no config, é: o desafiante precisa errar menos que a persistência e ter MAE pelo menos 2% menor que o do vigente. No primeiro ciclo, sem vigente, basta bater a persistência. O limiar existe para não trocar de modelo por diferenças que são só ruído de um dia a mais de dados.

Nos 3 ciclos completos houve uma promoção, no primeiro ciclo (quando ainda não existia modelo vigente), e o vigente foi mantido nos outros 2. No último ciclo (20261108-170654) o vencedor foi o modelo B (hist_gradient_boosting), com MAE de 1,897 °C na validação, contra 1,897 °C do vigente na mesma janela, uma melhora de 0,0% para um limiar de 2%. Decisão automática: **mantido**. A decisão, o motivo e as métricas ficam no run `decisao` do MLflow e no manifesto. Quando há promoção, o vigente antigo vai para a lista `anteriores` do manifesto, e todos os modelos continuam guardados no MLflow.

O vigente atual é o hist_gradient_boosting promovido no ciclo 20261107-205440 (dados `dv-20261024-622a2671`), treinado até 23/09/2025, com MAE de 1,887 °C na validação daquele ciclo e run_id `63182e0a1f344b449fc6b3baa7e97bb7`.

Arquitetura do fluxo único



Figura 2. O único workflow (`.github/workflows/pipeline_mlops.yml`) e onde cada etapa lê e grava.

Há um só workflow (*Previsão da máxima diária*) e um só job, com os passos em sequência: obter o código e o estado, instalar o `requirements.txt`, executar os notebooks 01 a 06, um por passo, com `jupyter nbconvert --execute`, escrever o resumo da execução na página do run, salvar o estado, abrir a issue de alerta se for o caso e publicar as evidências como artifact. Os gatilhos são o schedule (`17 11 * * *`, 08:17 em Brasília) e o `workflow_dispatch` para depuração. O `concurrency` impede duas execuções ao mesmo tempo.

Persistência entre execuções. Como o runner é descartado, o estado vive no próprio repositório: no fim de cada ciclo o workflow commita `data/raw`, `mlflow.db`, `mlartefatos/`, `models/` e `evidencias/`, e a execução seguinte começa do checkout des-

se commit. O manifesto e o modelo vigente são lidos dali, sem depender de arquivos do runner anterior. O mesmo pacote também é publicado como artifact baixável.

Falhas. O shell do Actions roda com `-e`: se qualquer notebook der erro, o job para antes do commit, nada é promovido e o vigente continua o mesmo. Se a API não tiver dia novo, o workflow encerra depois do notebook 01 sem treinar.

MLflow portátil. O MLflow 3.18 não aceita mais o armazenamento em pasta (`./mlruns`), então usamos SQLite (`mlflow.db`). Ele grava o caminho absoluto dos artefatos, o que quebraria o histórico ao sair do runner. Na criação do experimento trocamos esse caminho por um relativo (`mlartefatos`), e com isso o mesmo `mlflow.db` funciona no runner, na nossa máquina e na de quem clonar o repositório, inclusive com `mlflow ui`.

Evidências de execução

Até agora houve 1 execução disparada pelo próprio schedule, em 08/11/2026, e ela trouxe dia novo e fechou um ciclo completo. A Tabela 4 lista as execuções do workflow e o que cada uma produziu. Os logs ficam na aba Actions do repositório; os notebooks executados de cada ciclo ficam em `evidencias/ciclos/<cycle_id>/`, junto com o `ciclo.json`, as tabelas e o resumo do ciclo; e cada ciclo termina com um commit do bot que diz o gatilho, a data e a decisão.

Tabela 4. Execuções do workflow *Previsão da máxima diária* no GitHub Actions.

início (Brasília)	gatilho	run	resultado
07/11/2026 17:53	workflow_dispatch	42413212313	ciclo 20261107-205440: promovido
07/11/2026 17:58	workflow_dispatch	42413582266	ciclo 20261107-205914: mantido
07/11/2026 18:02	workflow_dispatch	42413887811	sem dados novos, encerrado após o 01
08/11/2026 14:05	schedule	42503222028	ciclo 20261108-170654: mantido
08/11/2026 15:22	workflow_dispatch	42509727985	sem dados novos, encerrado após o 01

Tabela 5. Histórico dos ciclos completos (MAE em °C; A, B e vigente na validação do ciclo, teste para o vigente).

cycle_id	gatilho	dados até	dias novos	feat.	MAE A	MAE B	venc.	MAE vig.	decisão	MAE teste	alerta
20261107-205440	manual	24/10/2026	6.141	8	1,929	1,887	B		promovido	2,209	não
20261107-205914	manual	31/10/2026	7	8	1,933	1,867	B	1,889	mantido	2,366	não
20261108-170654	agendado	01/11/2026	1	8	1,945	1,897	B	1,897	mantido	2,263	não

O MLflow tem 18 runs no experimento `n3-tmax-jaragua`, 6 por ciclo (votação, A, B, decisão, previsões e monitoramento), todos com as tags `cycle_id`, `data_version`, `etapa`

e `evento`. Para consultar: `mlflow ui --backend-store-uri sqlite:///mlflow.db` na raiz do projeto e o filtro `tags.cycle_id = '...'` ou `tags.etapa = 'treino'`.

Previsões e teste integrado

O notebook 05 carrega o vigente pelo MLflow. O objeto salvo é o Pipeline inteiro (imputação, padronização e modelo) e a lista de features vem do manifesto, então a previsão usa exatamente o pré-processamento do treino. A avaliação final é no teste, os 30 dias mais recentes com alvo conhecido, que não entraram em nenhuma escolha do ciclo. No último ciclo o vigente teve MAE de 2,26 °C no teste, contra 2,44 °C da persistência. Como o teste cobre só um mês, o MAE dele oscila com o tempo que fez naquele mês: um mês com muitas mudanças bruscas de temperatura é mais difícil para qualquer modelo, inclusive para a persistência. Pelo mesmo motivo o R^2 perde sentido em 30 dias, já que a variação da máxima dentro de um mês é pequena, e por isso ele fica de fora aqui.

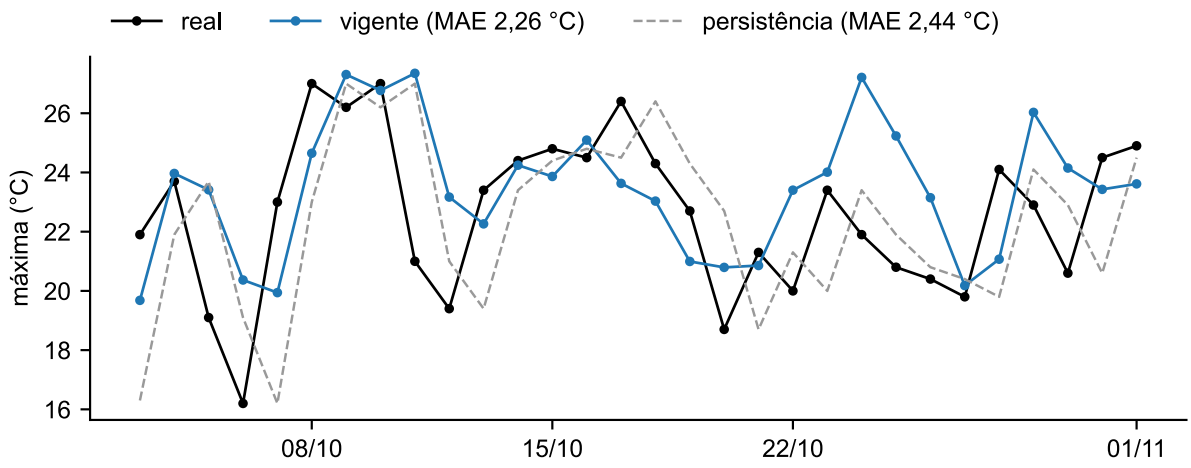


Figura 3. Avaliação final no teste do último ciclo: máxima real, prevista pelo vigente e persistência.

O notebook também prevê a máxima do dia seguinte ao último dado disponível e registra a previsão em `evidencias/previsoes_diarias.csv`. Quando o dado real daquele dia chega, num ciclo seguinte, o erro é preenchido. Das 3 previsões registradas até aqui, 2 já foram conferidas com o dado real, com erro absoluto médio de 2,86 °C.

Tabela 6. Previsões do dia seguinte feitas pelo pipeline.

data prevista	dados até	previsto (°C)	real (°C)	erro (°C)	cycle_id
25/10/2026	24/10/2026	25,2	20,8	4,4	20261107-205440
01/11/2026	31/10/2026	23,6	24,9	-1,3	20261107-205914
02/11/2026	01/11/2026	26,2		aguardando	20261108-170654

Teste integrado. No fim do notebook 05, asserts consultam o MLflow e conferem que o ciclo atual refez a votação, treinou A e B, que os dois usaram exatamente as

features eleitas, que todos os runs têm o mesmo `data_version` e que a decisão do ciclo está no manifesto. Se alguma condição falhar, o notebook dá erro e o workflow para. O mesmo notebook mostra o que mudou em relação ao ciclo anterior (dias novos, versão dos dados, features que entraram ou saíram, métricas e decisão).

Recurso adicional (critério 10): monitoramento de drift sazonal com alerta

Problema identificado. O vigente pode ficar muitos ciclos sem ser trocado (a regra de promoção é conservadora de propósito) e ninguém abre os notebooks todo dia. Precisávamos ser avisados quando os dados saem do normal (um mês muito mais quente ou seco que o habitual, ou uma mudança na própria fonte) e quando o modelo começa a errar mais.

Justificativa. A checagem de drift mais comum compara os dados recentes com o treino inteiro. Em clima isso dispara quase todo mês, porque novembro sempre é diferente da média do ano. Alarme que toca sempre vira ruído e passa a ser ignorado. O drift que interessa é o que foge do normal para a época.

Solução implementada. O notebook 06 (`src/monitoramento.py`) calcula o PSI de cada feature do vigente nos últimos 30 dias contra os dias da mesma época do ano (com 15 dias de margem) nos anos de treino do vigente, e também contra o ano todo, só para comparar. Também compara o MAE do vigente no último mês com o MAE dele nas mesmas datas do ano anterior, que estão na validação e ele nunca viu. Há alerta se alguma feature passar de PSI 1,0 ou se o erro passar de 1,5 vez o do ano anterior. Com só 30 dias o PSI tem um piso de ruído alto, então o limiar clássico de 0,25 dispararia demais; por isso rodamos a mesma checagem em todos os meses antigos do treino (backtest) e calibramos o limiar.

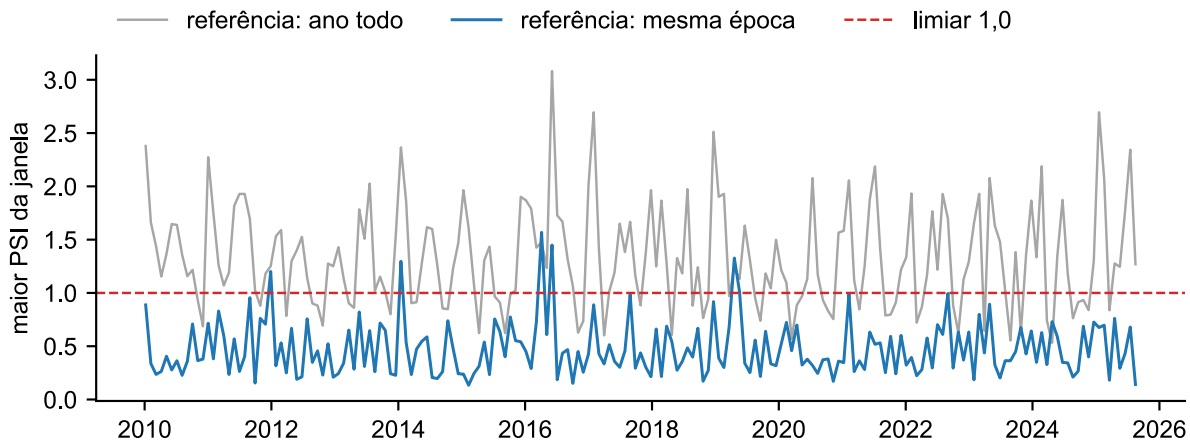


Figura 4. Backtest do alerta: maior PSI de cada janela de 30 dias do treino do vigente, com as duas referências.

Resultado demonstrado. No backtest, com o mesmo limiar, a referência do ano todo teria disparado alerta em 72% dos meses normais e a referência sazonal em 3%. No último ciclo o maior PSI sazonal foi 0,65 (contra 0,84 na comparação com o ano todo) e a razão de erro foi 1,21, então não houve alerta. Nos 3 ciclos não houve alerta. O canal de alerta foi testado de ponta a ponta com o input `testar_alerta` do workflow_dispatch, que abriu a issue #1 (Alerta de drift (teste do canal)) pelo próprio GitHub Actions.

Tabela 7. PSI das features monitoradas no último ciclo.

feature	PSI contra o ano todo	PSI contra a mesma época
`tmed`	0,79	0,33
`tmin`	0,84	0,17
`radiacao`	0,21	0,28
`vento_dir_cos`	0,27	0,04
`tmax`	0,59	0,65
`pressao_delta`	0,07	0,16
`rajada`	0,12	0,33

Ponto de integração. É o último notebook do mesmo workflow, depois das previsões. As métricas vão para o run `monitoramento` do MLflow e para o `ciclo.json`; se houver alerta, o notebook grava `alerta.md` e o passo seguinte do YAML abre a issue "Alerta de drift" no repositório (ou comenta na que já estiver aberta), o que manda notificação para quem acompanha o repositório.

Limitações e conclusões

Limitações. Os dados são de reanálise e de um ponto só, com atraso de uma semana, então o modelo não serve para a previsão de amanhã no calendário sem uma fonte observada em tempo real. Os hiperparâmetros são fixos no config, sem busca. O teste de 30 dias é pequeno e oscila com o tempo que fez no mês. Guardar o `mlflow.db` e os artefatos no Git funciona bem para um trabalho, mas o repositório cresce uns 2,5 MB por ciclo (quase tudo é o modelo B serializado); num projeto real usaríamos um servidor de tracking e um armazenamento de objetos. E o GitHub pode atrasar ou pular execuções agendadas em horários de pico.

Conclusões. O pipeline roda sozinho de ponta a ponta: cada execução agendada busca dados reais novos, gera uma versão nova dos dados, refaz a votação, treina os dois modelos com as mesmas features, compara com o vigente numa janela que nenhum dos dois viu e decide automaticamente, deixando tudo rastreável no MLflow, no manifesto e no histórico do Git. Os modelos erram cerca de 27% menos que a persistência. A maioria dos ciclos mantém o vigente, o que é o comportamento espe-

rado: um dia a mais de dados quase não muda o problema, e o limiar de promoção evita trocar de modelo por ruído. O monitoramento sazonal resolveu um problema que só apareceu quando olhamos os dados de perto: a checagem tradicional de drift não serve para clima.